Data Analytics and Mining

Intro to Machine Learning, Supervised Learning

Data Analytics and Mining, 2026

Fedor Shlyakhov
National Research University Higher School of Economics



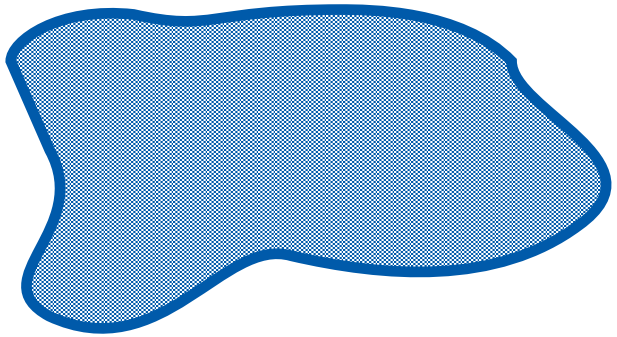
October 23, 2026

Supervised Learning



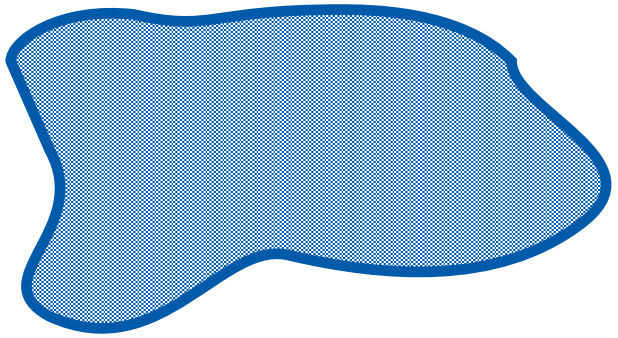
Problem setup

$\mathcal{X}$ – a set of objects

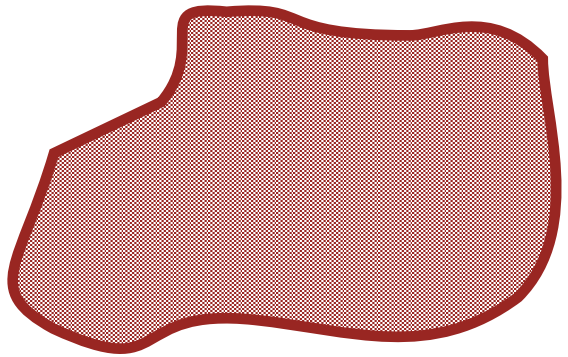


Problem setup

x – a set of objects



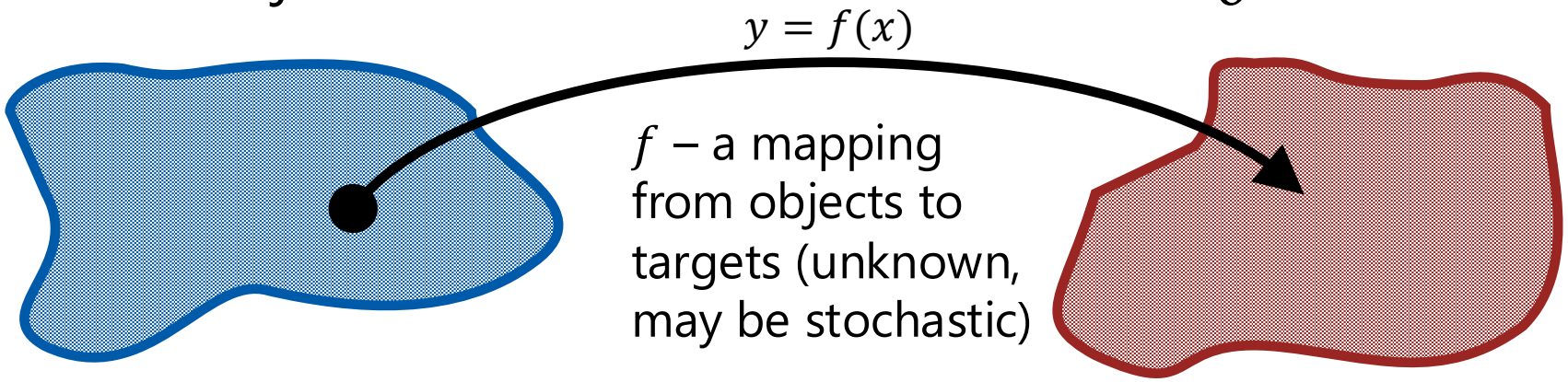
y – a set of targets



Problem setup

x – a set of objects

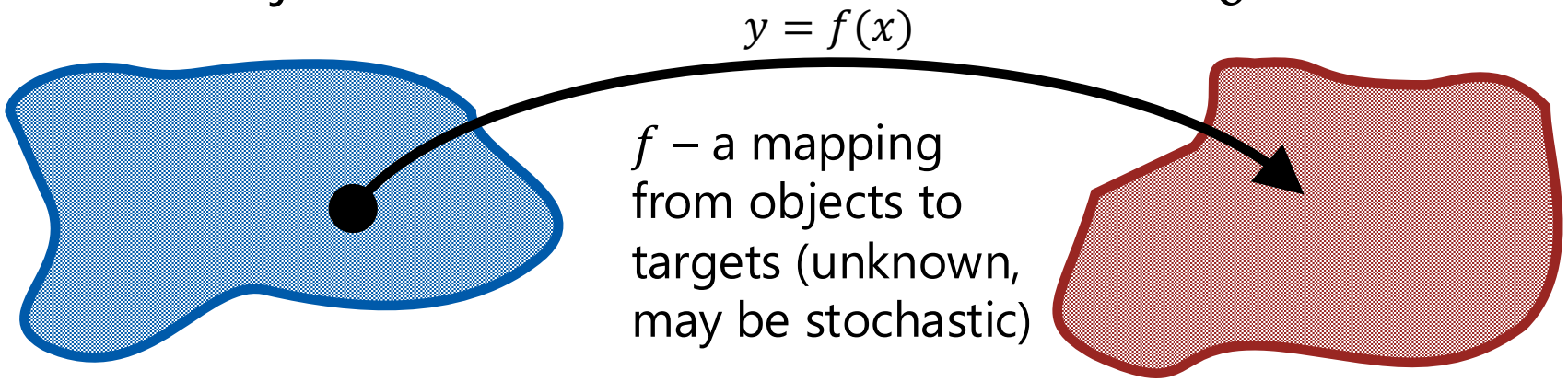
y – a set of targets



Problem setup

$\mathcal{X}$ – a set of objects

$\mathcal{Y}$ – a set of targets



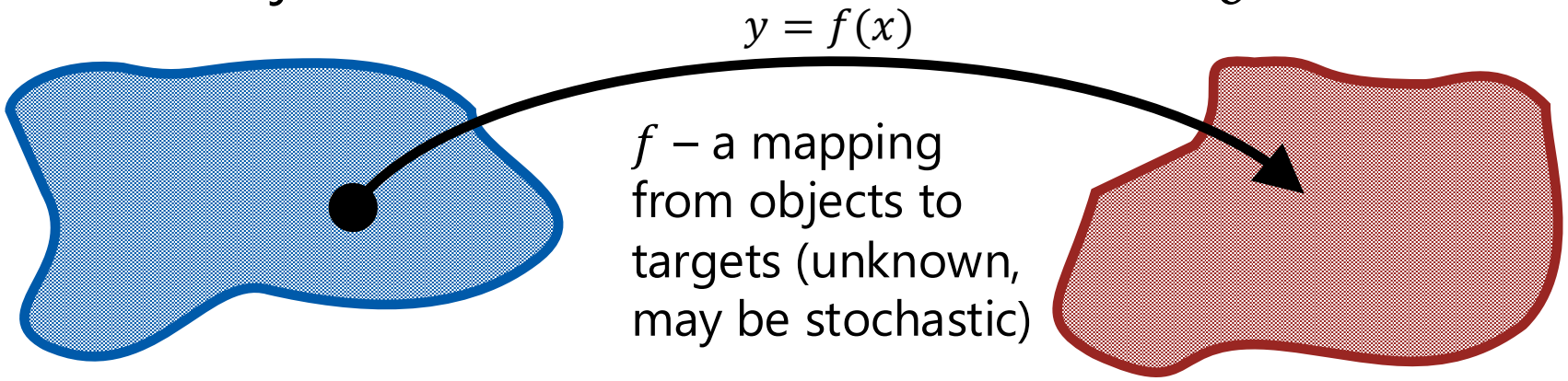
A dataset: $D = \{(x_i,\ y_i) : i = 1,\ 2, \dots,\ N\}$

$$x_i \in \mathcal{X}, \qquad y_i = f(x_i) \in \mathcal{Y}$$

Problem setup

$\mathcal{X}$ – a set of objects

$\mathcal{Y}$ – a set of targets



A dataset: $D = \{(x_i,\ y_i) : i = 1,\ 2, \dots,\ N\}$

$$x_i \in \mathcal{X}, \qquad y_i = f(x_i) \in \mathcal{Y}$$

Goal: **approximate f given D**
i.e. learn to **recover targets from objects**

Examples

- Iris flower species classification

Objects

Individual flowers, described by the length and width of their sepals and petals

Targets

Species to which this particular flower belongs

Mapping

Different shapes of sepals and petals correspond to different species

(non-deterministic)





images source: wikipedia.org

Examples

- Spam filtering

Objects

E-mails (sequences of characters)



Targets

“spam” / “not spam”

Mapping

Message content defines whether it’s spam or not

(non-deterministic, varies from person to person)

Icon made by Freepik from www.flaticon.com

Examples

- CAPTCHA recognition

Objects

CAPTCHA images (vectors of pixel brightness levels)

Targets

Sequences of characters

Mapping

Inverse of CAPTCHA generating algorithm

(almost deterministic, depending on the level of distortion)

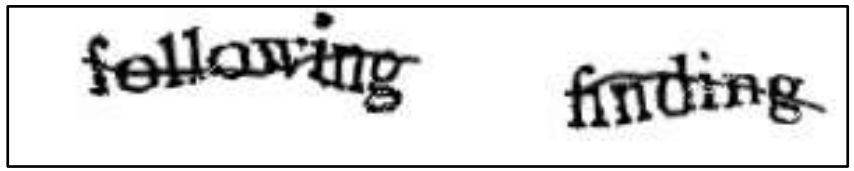


image source: wikipedia.org

Learning check

1. In supervised learning, a dataset D consists of …

a. only objects x_i

b. pairs (x_i, y_i) of objects and their targets

c. only targets y_i

d. the true mapping f

2. In spam filtering, what is the target?

a. the text of the e-mail

b. the address of the sender

c. "spam" / "not spam"

d. the number of characters

3. The goal of a supervised learning algorithm is to …

a. memorize the dataset

b. approximate the unknown mapping f from objects to targets

c. collect more data

d. delete strange objects

4. In CAPTCHA recognition, what are the objects?

a. sequences of characters

b. CAPTCHA images (pixel brightness levels)

c. the algorithm that generates CAPTCHAs

d. e-mails

Features



Features

- Objects x_i are described by features x_i^j, i.e.:
  - It's a vector $x_i = (x_i^1, x_i^2, \dots, x_i^d)$

Features

- Objects x_i are described by features x_i^j, i.e.:
  - It's a vector $x_i = (x_i^1, x_i^2, \dots, x_i^d)$
- many algorithms require that the dimensionality d of the data is same for all objects

Features

- Objects x_i are described by features x_i^j, i.e.:
  - It's a vector $x_i = (x_i^1, x_i^2, \dots, x_i^d)$
- many algorithms require that the dimensionality d of the data is same for all objects
  - In such case the objects may be organised in a design matrix:

$$X = \begin{bmatrix} x_1^1 & x_1^2 & \cdots & x_1^d \\ x_2^1 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \ddots & \vdots \\ x_N^1 & x_N^2 & \cdots & x_N^d \end{bmatrix}$$

features (→)

objects (↓)

Example: Iris dataset

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
5.1	3.5	1.4	0.2
4.9	3.0	1.4	0.2
4.7	3.2	1.3	0.2
4.6	3.1	1.5	0.2
5.0	3.6	1.4	0.2
...	...	...	...
6.7	3.0	5.2	2.3
6.3	2.5	5.0	1.9
6.5	3.0	5.2	2.0
6.2	3.4	5.4	2.3
5.9	3.0	5.1	1.8

In this example, all featuers are real numbers

Feature types

- Individual features x_i^j may be of various nature
- Common cases:
  - **Numeric features**, e.g.:
    - Sepal length
    - Height of a building
    - Temperature
    - Price
    - Age
    - Etc.

Feature types

- Individual features x_i^j may be of various nature
- Common cases:
  - **Categorical**

nominal (no order implied), e.g.:

Color
City of birth
Name

ordinal (values can be compared, though pairwise differences are not defined), e.g.:

Level of education
Age category (child, teen, adult, etc.)

Feature types

- Individual features x_i^j may be of various nature
- Common cases:
  - **Binary**, e.g.:
    - True / False
  - Can be treated as numeric (0/1 or –1/+1)

One-hot encoding

- How does one convert categorical feature to numeric?

One-hot encoding

- How does one convert categorical feature to numeric?
  - Assigning each category a number (e.g. ”red” = 1, “green” = 2, etc.) may have negative effect on the learning algorithm

One-hot encoding

- How does one convert categorical feature to numeric?
  - Assigning each category a number (e.g. "red" = 1, "green" = 2, etc.) may have negative effect on the learning algorithm
- One-hot encoding – simple trick to convert categorical feature to numeric:

color		is_blue	is_red	is_green
"red"		0	1	0
"red"		0	1	0
"blue"	→	1	0	0
"green"		0	0	1
"blue"		1	0	0

A trick for ordinal features

- One-hot encoding may be used, though it loses the information about the relations between the categories

A trick for ordinal features

- One-hot encoding may be used, though it loses the information about the relations between the categories
- Similar trick:

Academic degree	is_bachelor	is_master	is_PhD
"none"	0	0	0
"bachelor"	1	0	0
"master"	1	1	0
"PhD"	1	1	1
"master"	1	1	0

More advanced encoding techniques

- See https://contrib.scikit-learn.org/category_encoders/index.html

Learning check

1. Which feature is nominal?

a. age in years

b. level of education

c. city of birth

d. temperature

2. Which feature is ordinal?

a. colour

b. age category (child, teen, adult)

c. name

d. price

3. One-hot encoding with columns (is_blue, is_red, is_green). How is "green" encoded?

a. (0, 1, 0)

b. (0, 0, 1)

c. (1, 1, 0)

d. 3

4. Why is red = 1, green = 2, blue = 3 a bad encoding?

a. numbers take too much memory

b. it adds an order (green is "between" red and blue) that does not exist

c. algorithms cannot read numbers

d. it is a perfect encoding

Learning Algorithms



Machine Learning Algorithm

Algorithm $\mathcal{A}$:

given a dataset $D = \{(x_i,\ y_i) : i = 1,\ 2, \dots,\ N\}$

$$x_i \in \mathcal{X},\ y_i = f(x_i) \in \mathcal{Y}$$

returns an approximation $\hat{f} = \mathcal{A}(D)$ to the true dependence f.

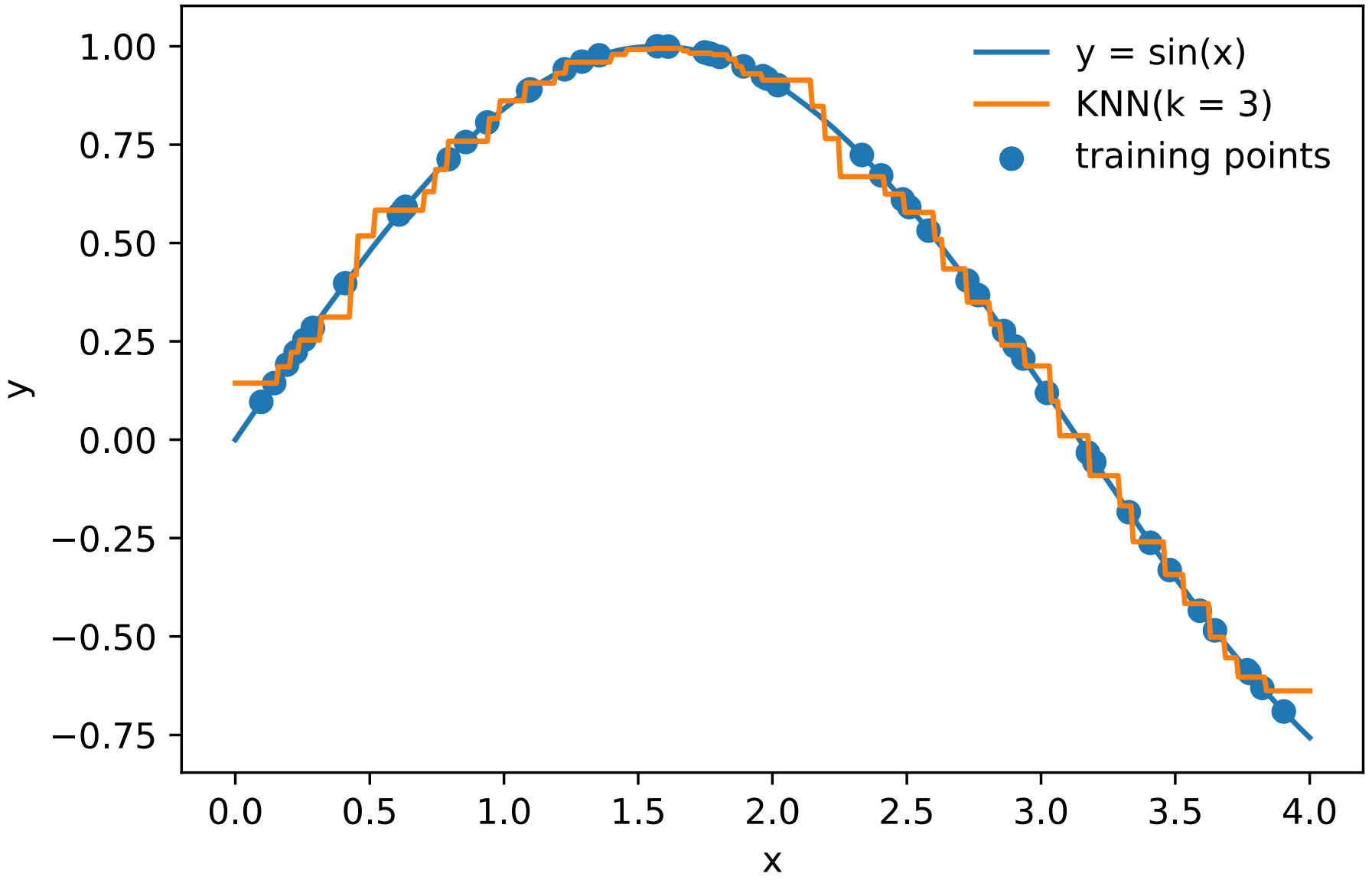
Example: k nearest neighbors (kNN)

- Idea: close objects should have similar targets
- Why don't we look up k closest (by some metric of the feature space) objects in the dataset and average their targets:

$$\hat{f}(x) = \frac{1}{k} \sum_{i:\, x_i \in D_x^k} y_i$$

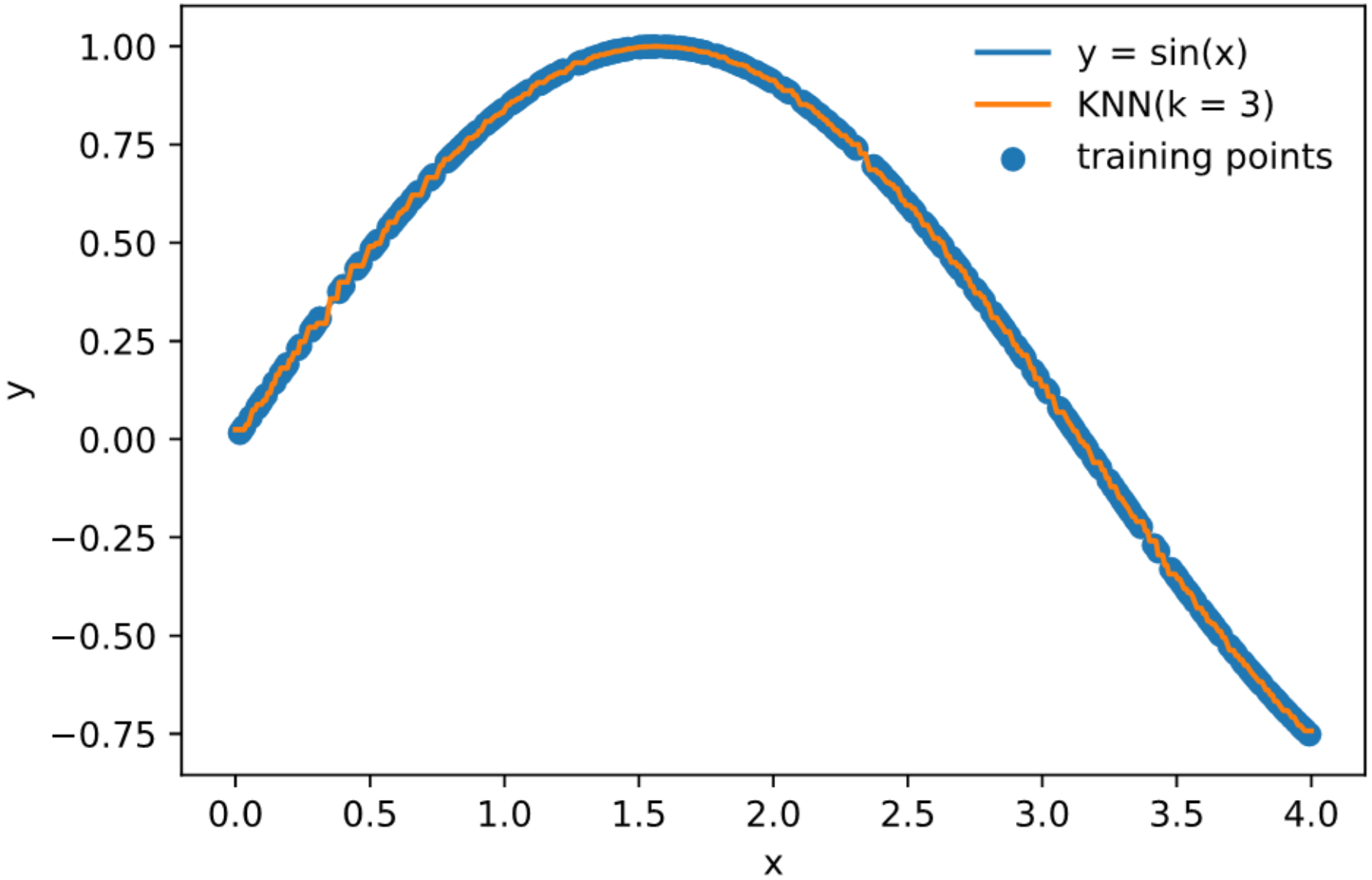
D_x^k – set of k objects from D closest to x

Example: k nearest neighbors



training points: 50

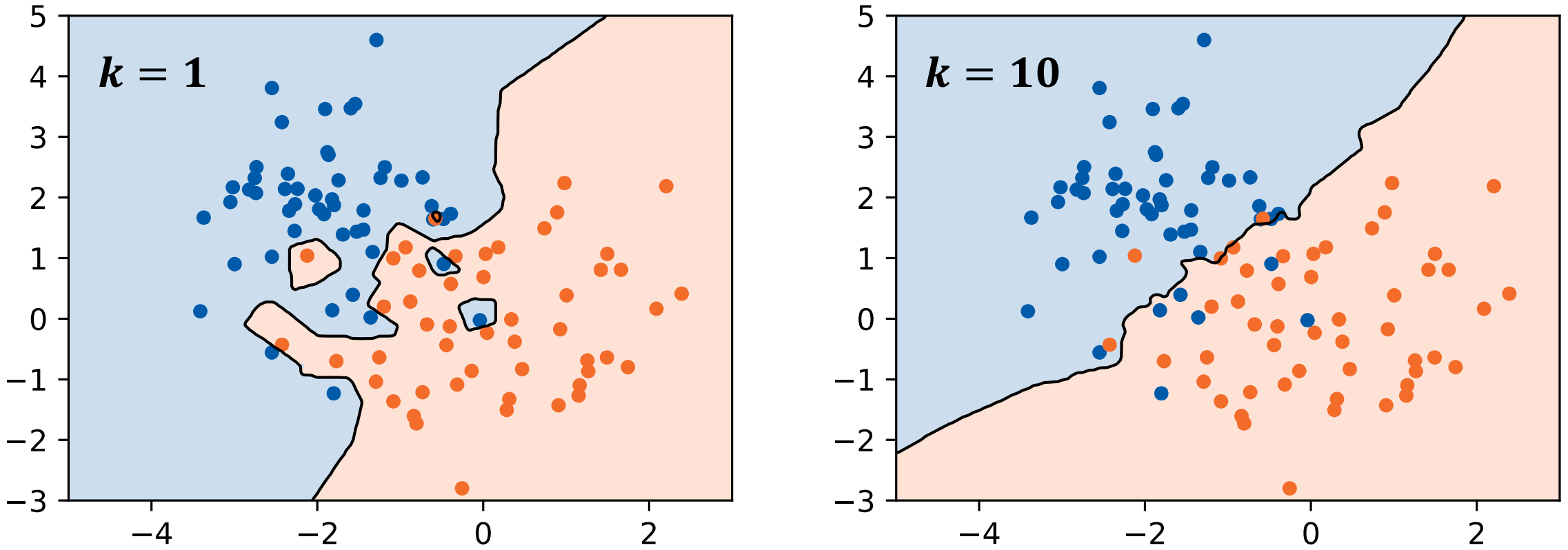
Example: k nearest neighbors



training points: 250

More data = better approximation

Example: k nearest neighbors



$$\hat{f}(x) = \underset{C}{\operatorname{argmax}} \sum_{i:\, x_i \in D_x^k} \mathbb{I}[y_i = C]$$

Classification example

D_x^k – set of k objects from D closest to x

Loss function

- How does an algorithm find the approximation $\hat{f} = \mathcal{A}(D)$ to the true mapping function?

Loss function

- How does an algorithm find the approximation $\hat{f} = \mathcal{A}(D)$ to the true mapping function?
- Many algorithms work by solving an **optimization task**

Loss function

- How does an algorithm find the approximation $\hat{f} = \mathcal{A}(D)$ to the true mapping function?
- Many algorithms work by solving an **optimization task**
- We can measure the quality of a prediction for a single object x_i with a **loss function** $\mathcal{L} = \mathcal{L}\left(y_i, \hat{f}(x_i)\right)$

E.g. squared error:

$$\mathcal{L} = \left(y_i - \hat{f}(x_i)\right)^2$$

Loss function

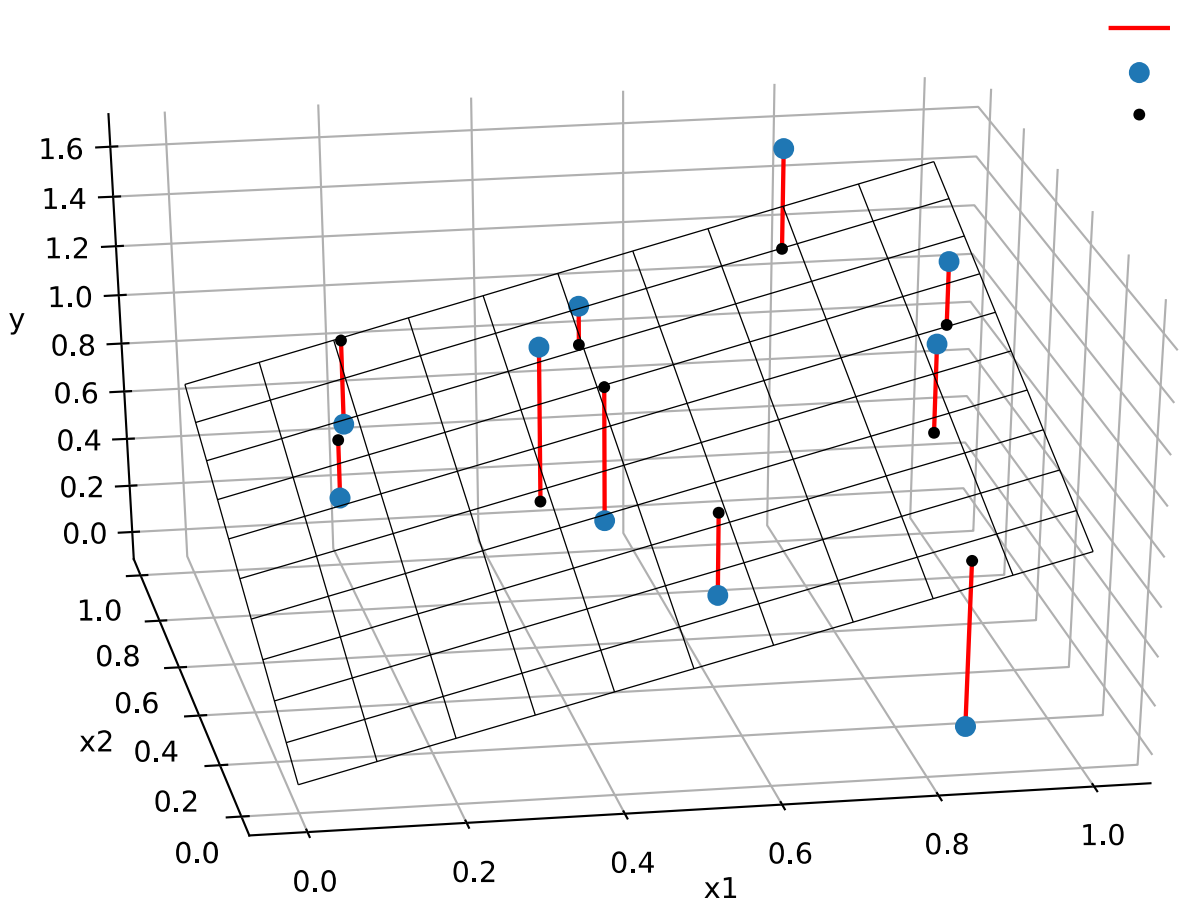
- How does an algorithm find the approximation $\hat{f} = \mathcal{A}(D)$ to the true mapping function?
- Many algorithms work by solving an **optimization task**
- We can measure the quality of a prediction for a single object x_i with a **loss function** $\mathcal{L} = \mathcal{L}\left(y_i, \hat{f}(x_i)\right)$
- Then, learning (or training) can be formulated as a **loss minimization** problem:

$$\hat{f} = \underset{\tilde{f}}{\operatorname{argmin}} \underset{(x,y) \in D}{\mathbb{E}} \mathcal{L}\left(y, \tilde{f}(x)\right)$$

E.g. squared error:

$$\mathcal{L} = \left(y_i - \hat{f}(x_i)\right)^2$$

Example: linear regression



$$\hat{f}_{w,b}(x) = w^{\mathrm{T}}x + b$$

$$w \in \mathbb{R}^d$$
$$b \in \mathbb{R}$$
$$x \in \mathcal{X} \subset \mathbb{R}^d$$

$$\frac{1}{N}\sum_{i=1\ldots N}\left(y_i - \hat{f}_{w,b}(x_i)\right)^2 \longrightarrow \min_{w,b}$$

Mean Squared Error (MSE loss)

Learning check

1. kNN regression, k = 3. The 3 nearest neighbours of x have targets 2, 4 and 9. The prediction is …

a. 4

b. 5

c. 9

d. 15

2. kNN classification, k = 5. The 5 nearest neighbours belong to classes A, A, B, A, B. The prediction is …

a. A

b. B

c. a tie

d. impossible to say

3. Squared error: the true target is y = 10, the prediction is 7. The loss is …

a. 3

b. −3

c. 9

d. 17

4. Training a model usually means …

a. maximizing the loss

b. minimizing the average loss on the training data

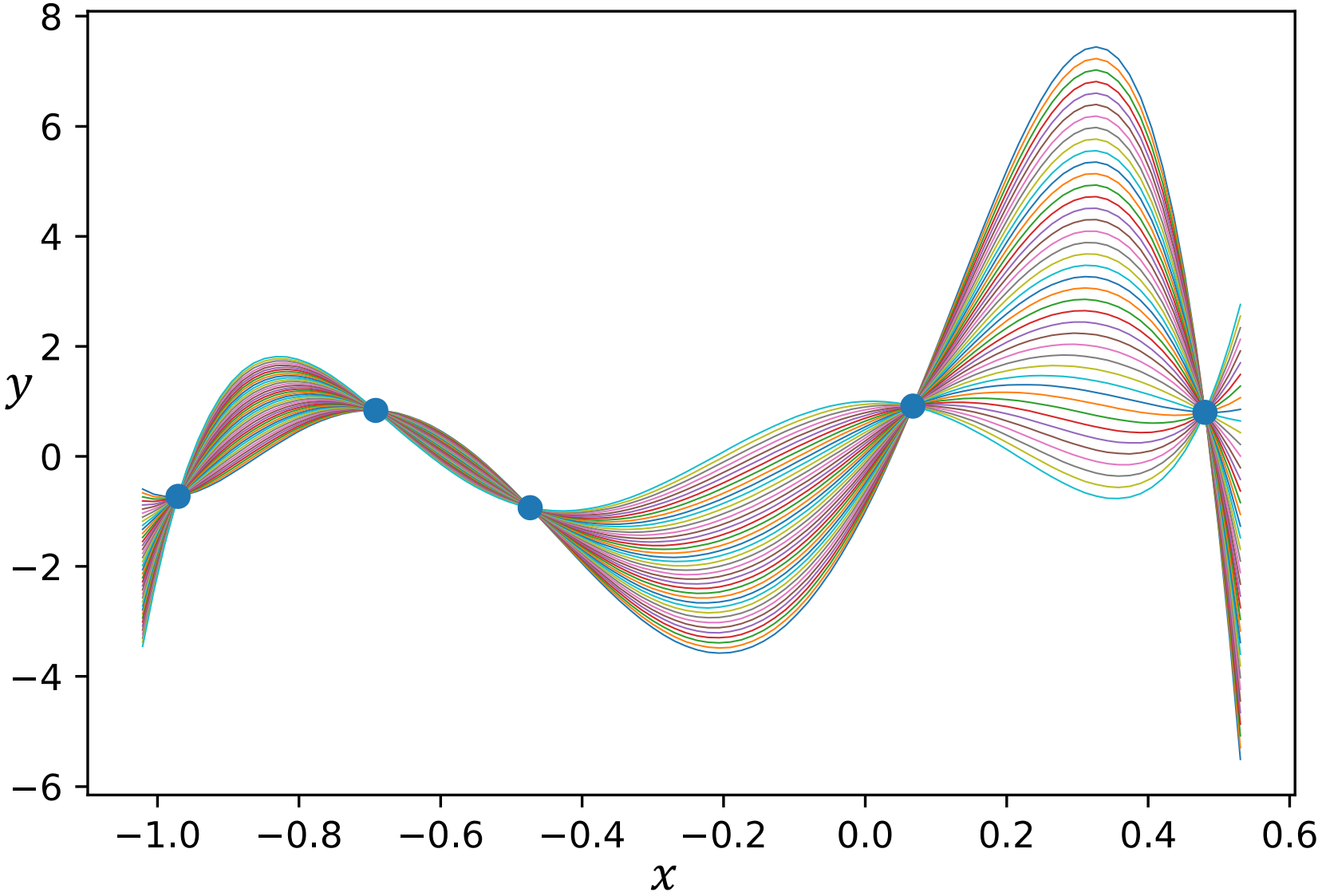
c. choosing k at random

d. removing features

Assumptions about data



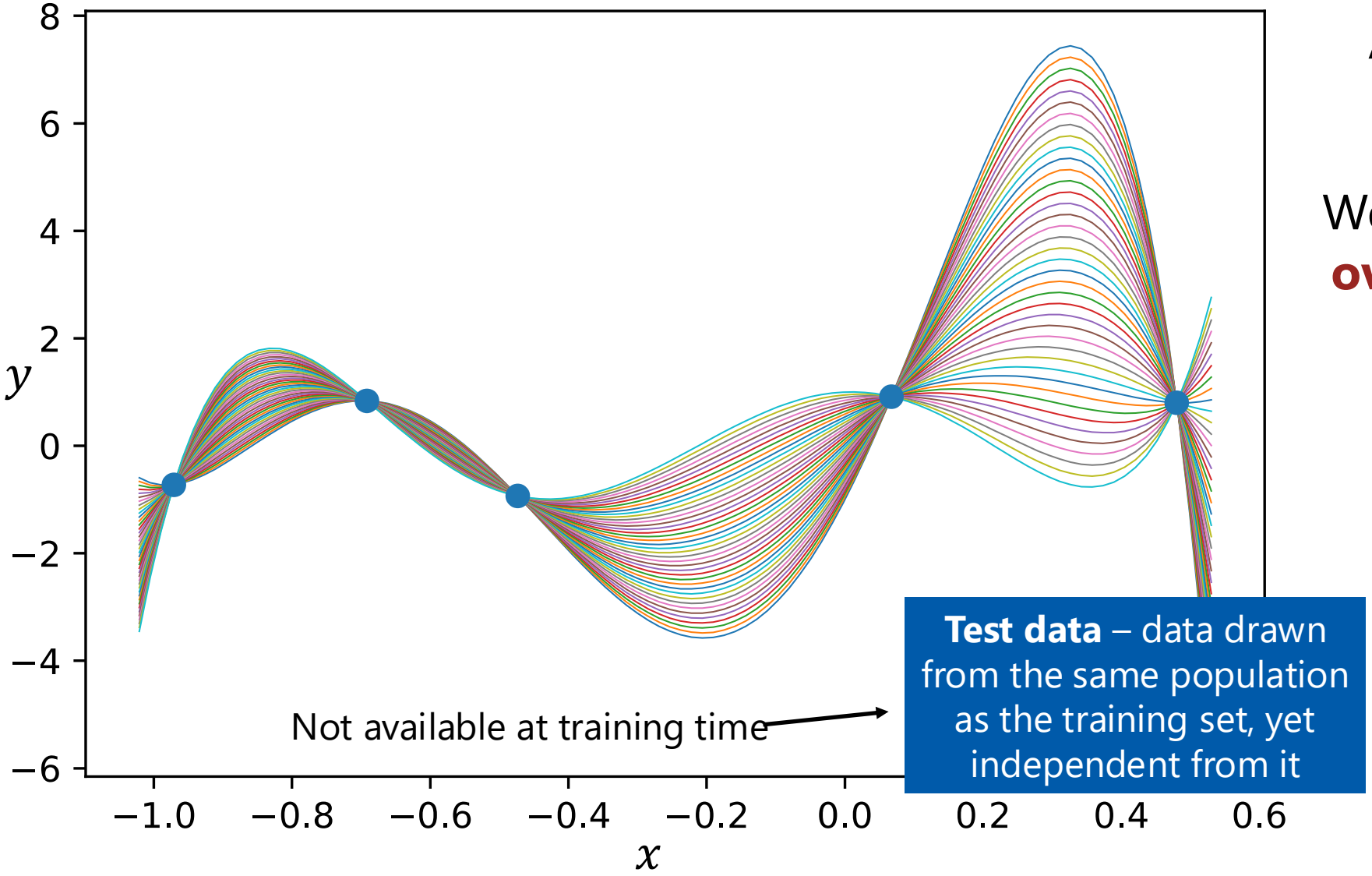
No assumptions = Infinitely many solutions



Any of these curves minimizes the loss

We want **expected loss over population** to be minimized

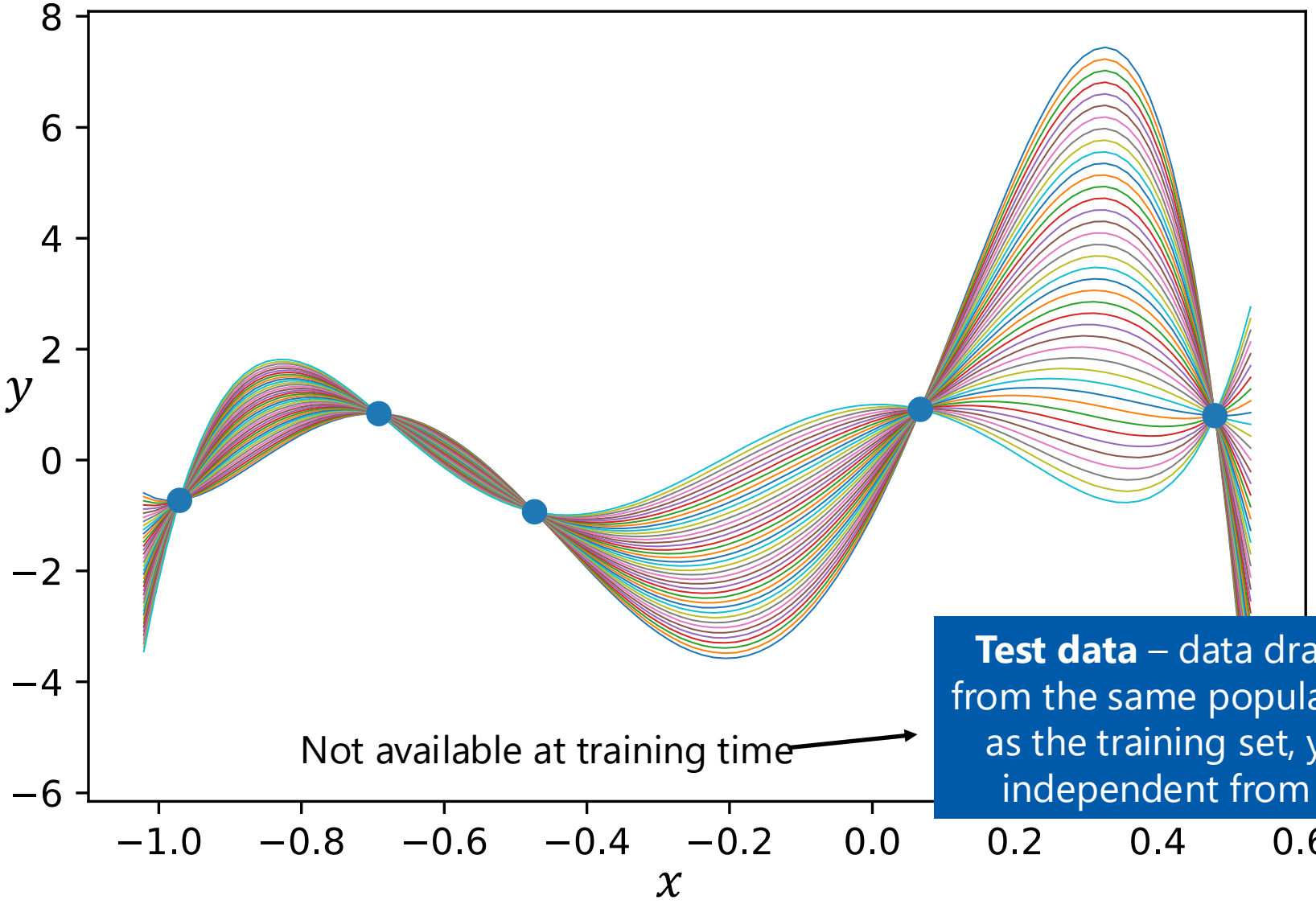
No assumptions = Infinitely many solutions



Any of these curves minimizes the loss

We want **expected loss over population** to be minimized

No assumptions = Infinitely many solutions

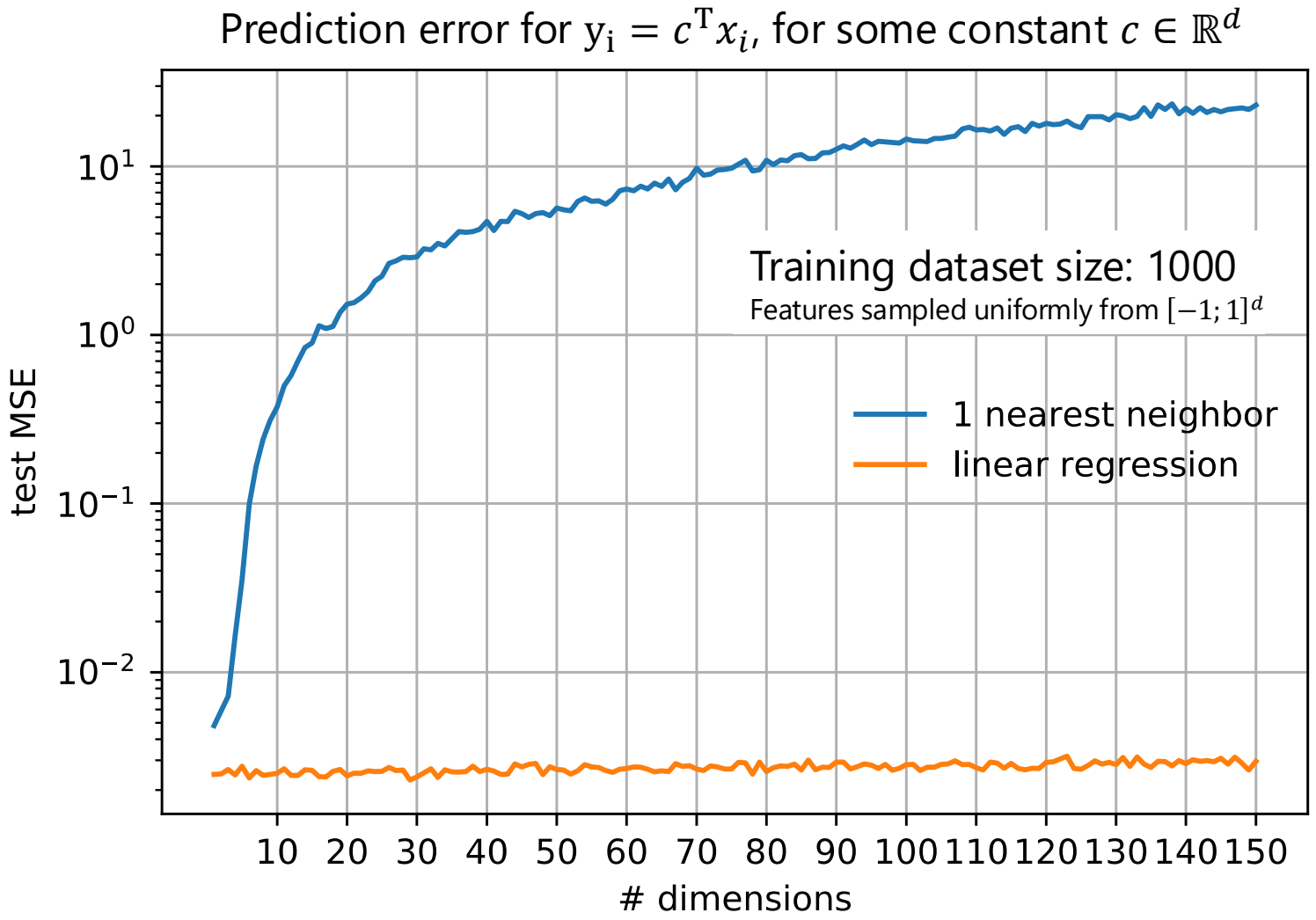


Any of these curves minimizes the loss

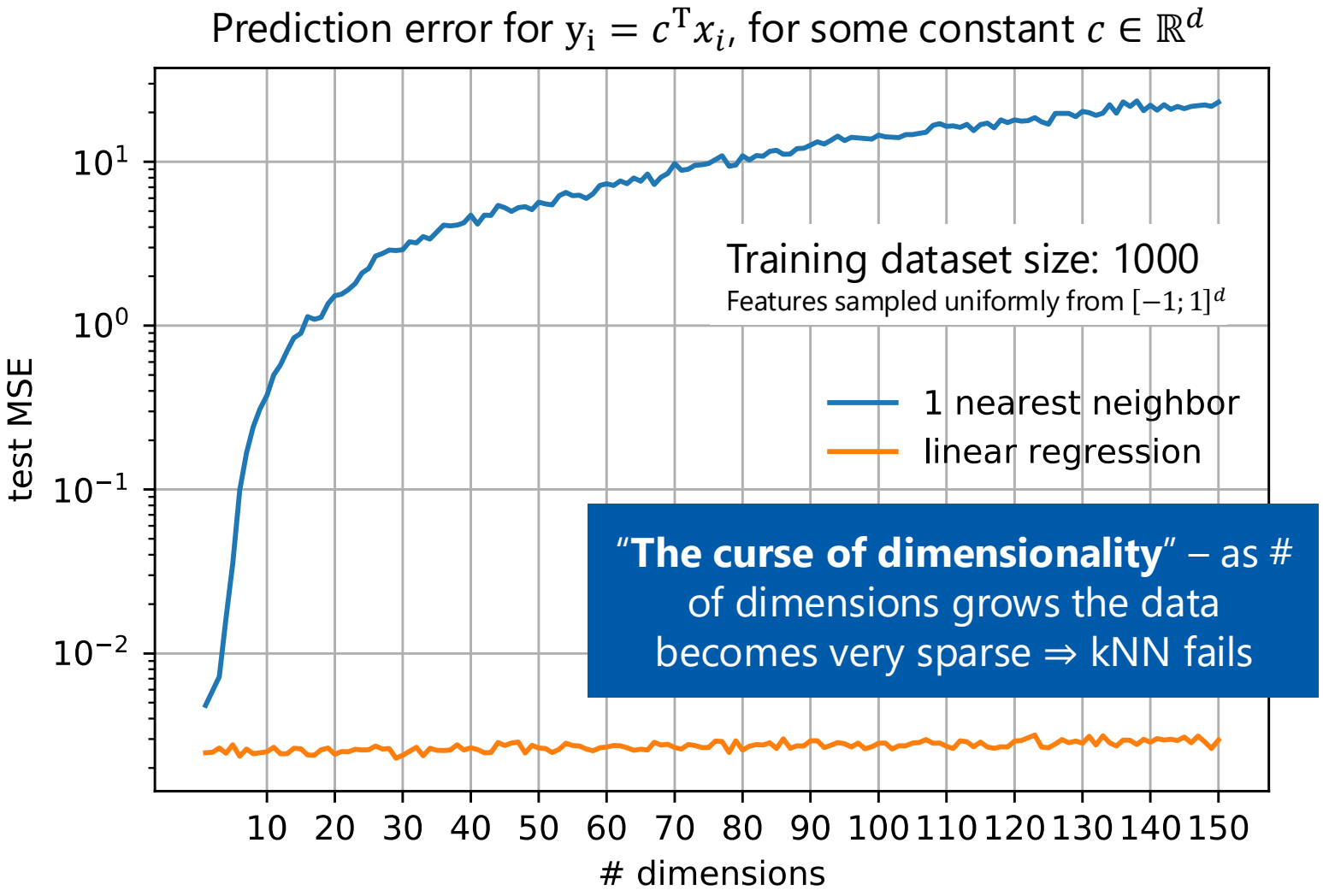
We want **expected loss over population** to be minimized

Need to **assume some structure** of the data, common to **training** and **testing** data

Example: kNN(k = 1) VS Linear Regression

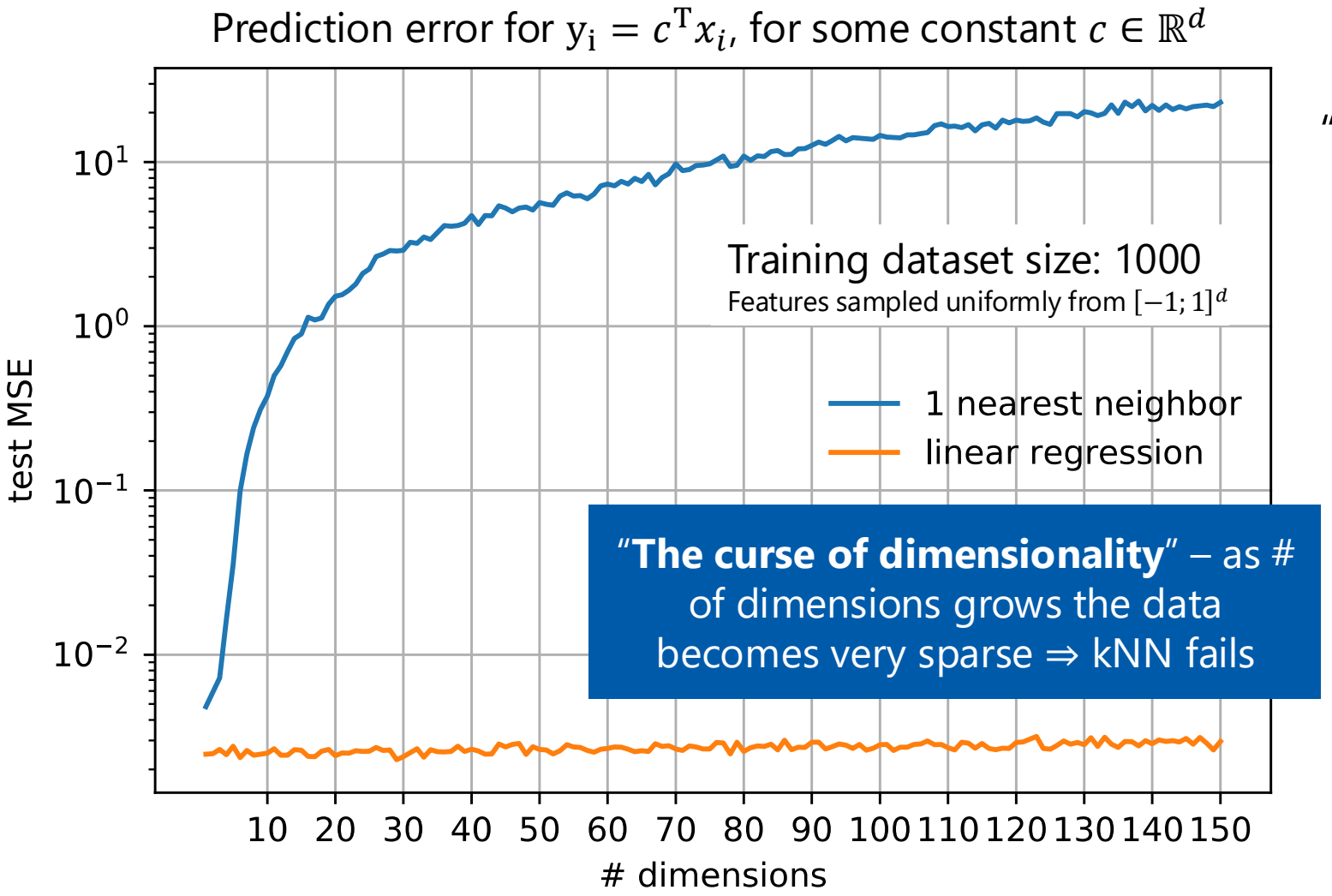


Example: kNN(k = 1) VS Linear Regression



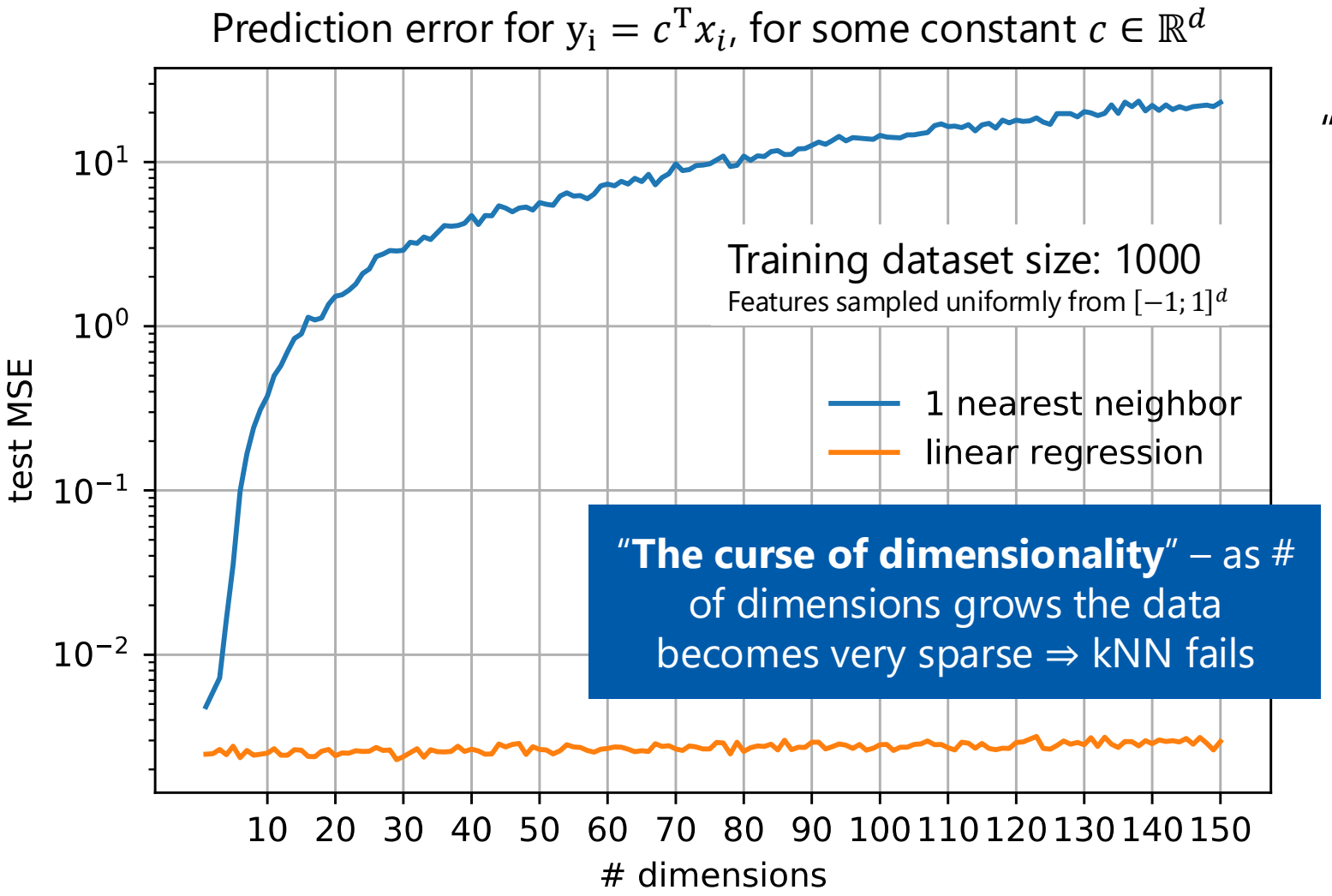
"**The curse of dimensionality**" – as # of dimensions grows the data becomes very sparse ⇒ kNN fails

Example: kNN(k = 1) VS Linear Regression



Assumption for kNN:
"**similar objects have similar targets**"

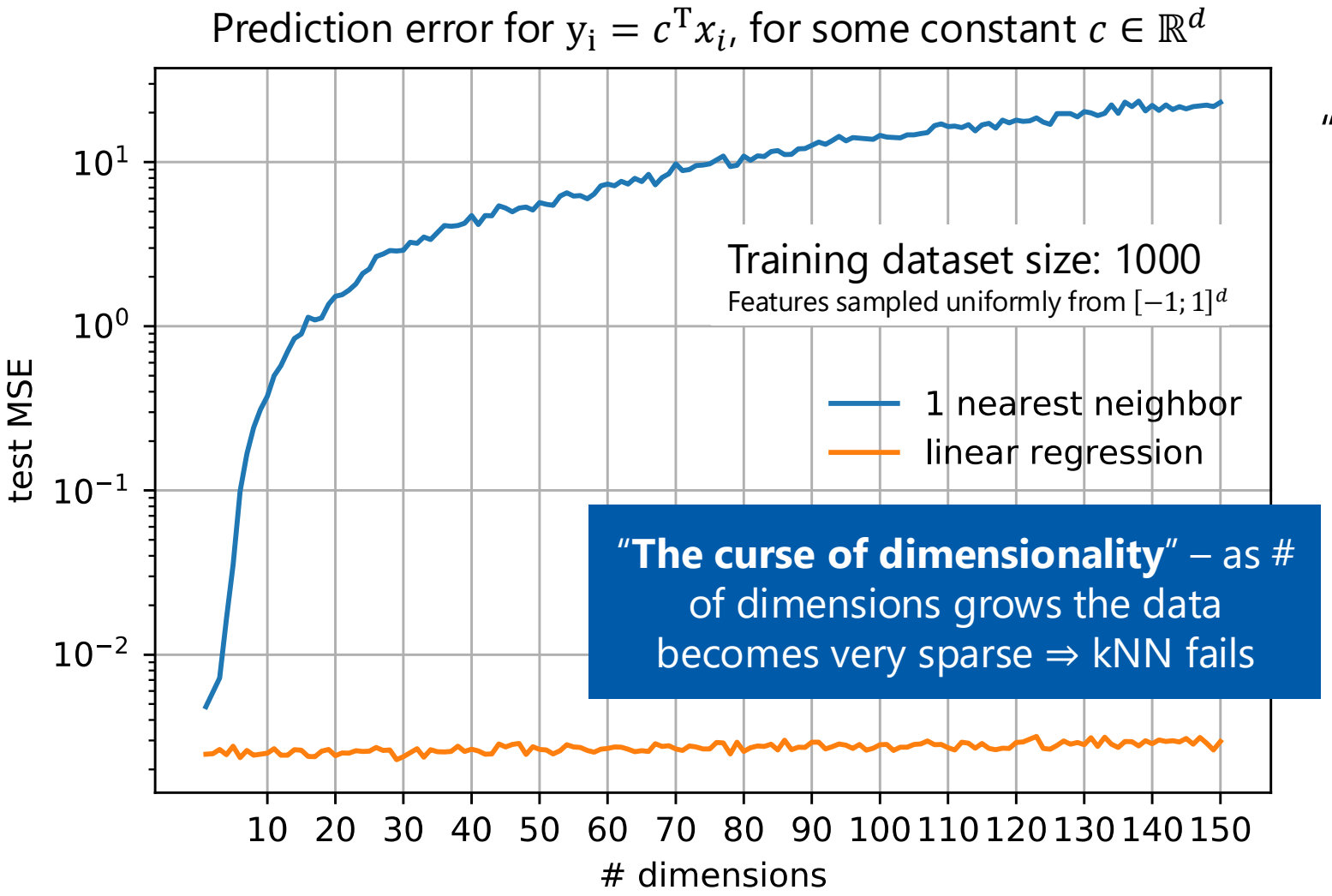
Example: kNN(k = 1) VS Linear Regression



Assumption for kNN:
"**similar objects have similar targets**"

Assumption for Linear Regression:
"**targets are linear in features**"

Example: kNN(k = 1) VS Linear Regression

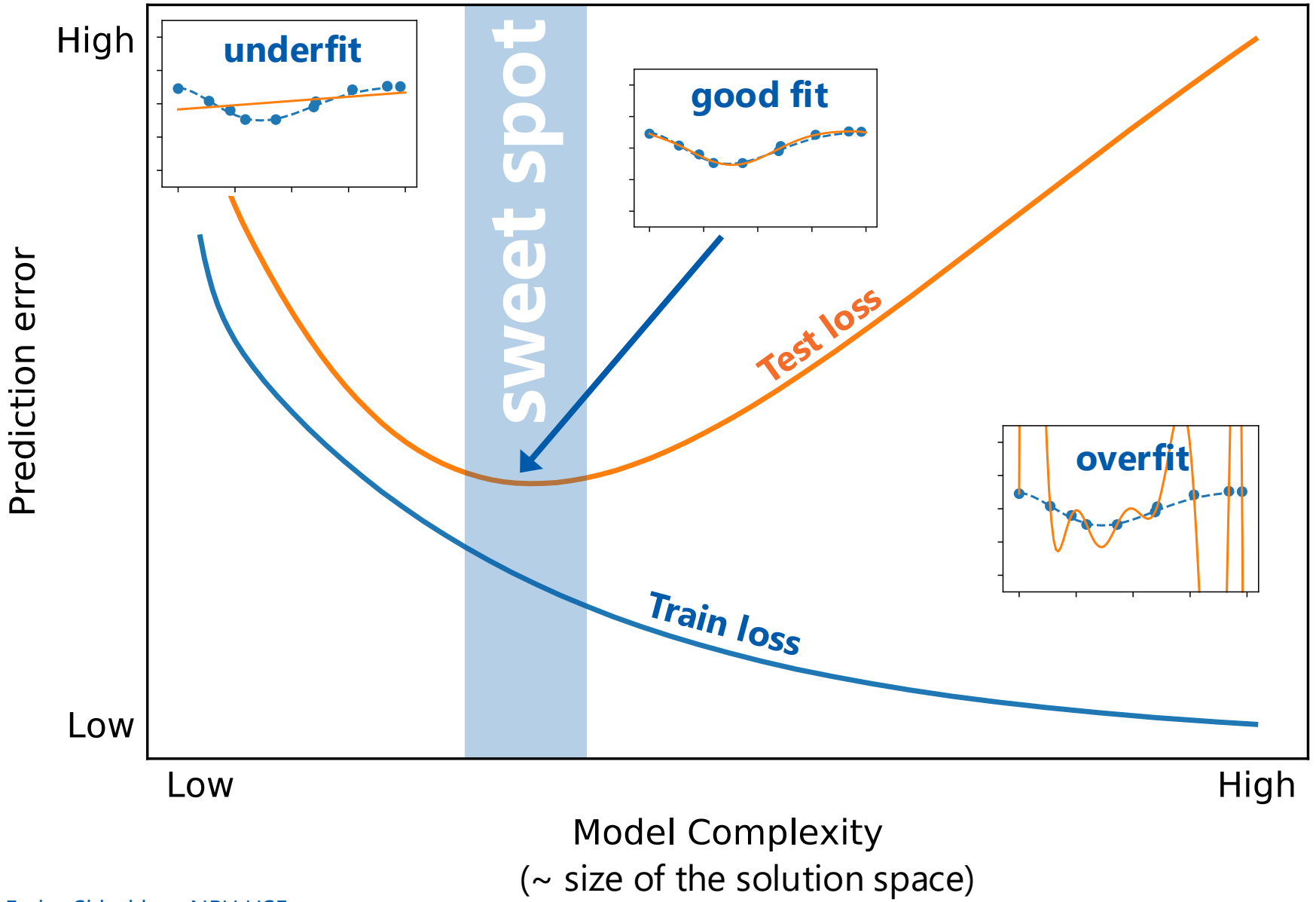


Assumption for kNN:
"**similar objects have similar targets**"

Assumption for Linear Regression:
"**targets are linear in features**"

For this example, both assumptions are correct, but one is **stronger** than the other

How to check whether a model is good?



Check the loss on the **test data** – i.e. data that the learning algorithm "hasn't seen"

The goal is to find the **right level of limitations** – not too strict, not too loose

Learning check

1. A model has a very low train loss but a high test loss. This is …

a. underfitting

b. overfitting

c. a good fit

d. impossible

2. Why do we need test data?

a. to train the model faster

b. to see how the model works on new data it has not seen

c. to make the train loss smaller

d. we do not need it

3. kNN classification: which is more likely to overfit?

a. k = 1

b. k = 10

c. both the same

d. it depends on the colours

4. A straight line fitted to data that clearly follow a U-shape is an example of …

a. overfitting

b. underfitting

c. a good fit

d. one-hot encoding

Summary

- Supervised Machine Learning algorithms build approximations $\hat{f} = \mathcal{A}(D)$ to the true dependence f
- Features may be of various nature, one-hot encoding is useful to convert categorical features to numeric
- Machine Learning algorithms can be defined as expected loss minimization tasks
- Choosing the right model = applying the right assumptions about the data
- Use test data to detect underfitting and overfitting